KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Digital Wallet and Balance Management System

Submitted by

JANARTHANAN MS

25BCS121

Under the Guidance of

Mr.ARAVIND

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled " RetailShopManagement " submitted in partial fulfillment of the requirements for the course **JAVA PROGRAMMING** is a **record of my original work** carried out under the guidance of **Mr.ARAVIND**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

JANARTHANAN MS

CERTIFICATE

This is to certify that the project entitled **"Digital Wallet and Balance Management System"** is a **bona fide work** carried out by JANARTHANAN MS **I "B"/2025**, **BSC COMPUTER SCIENCE, KG College of Arts & Science**, in partial fulfillment of the requirements for the course **JAVA PROGRAMMING**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: COIMBATORE

Date:10/04/2026

Submitted for the Viva-Voce Examination held on 10/04/2026

Guide:	**Head of the Department**
(Technical Trainer, KGCAS)	(Technical Head, KGCAS)
Mr.Aravind	Mr.Boopalan Sundaramoorthy

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	**7**
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	**9**
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	**11**
3.1	Modules	
4	**Methodology**	**13**
4.1	Problem Analysis	

4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	**16**
6	**Input and Output Screens**	**19**
7	**Discussion and Conclusion**	**21**
7.1	Discussion	
7.2	Conclusion	
8	**References**	**22**

ABSTRACT

Thegital Wallet and Balance Management System is a Java-based application designed to manage electronic transactions efficiently. In many situations, people find it difficult to track their spending and balance manually, which may lead to calculation errors and poor financial management. This project aims to provide a simple and automated solution for managing digital money.

The system allows users to add money to their wallet, spend money for various purposes, and check their current balance at any time. All transactions are processed automatically, ensuring accuracy and reducing human errors. The application is developed using Core Java programming concepts such as classes, objects, methods, conditional statements, loops, and exception handling. It can be implemented as a console-based application or enhanced using Java Swing for a graphical user interface.

The main objective of this system is to provide an easy and efficient way to handle digital transactions, improve accuracy in balance calculation, and save time. The system displays clear information about transactions and the available balance, helping users maintain better financial control.

1.INTRODUCTION

The Retail Shop Management System is a Java-based application designed to manage the daily activities of a retail shop. It helps the shopkeeper maintain product details, check stock availability, record sales, and calculate customer bills.

In a traditional retail shop, product details and sales records are often maintained manually. This method can lead to calculation errors, misplaced records, and difficulty in tracking available stock.

The proposed system uses Java programming and object-oriented programming concepts to organize shop operations in a simple and efficient manner. It reduces manual work, improves accuracy, and makes product and sales management easier.

1.1 Objective

- The main objectives of the Retail Shop Management System are:

- To maintain product details such as product ID, name, price, and quantity.
- To add new products and update existing product information.
- To check the availability of products.
- To record customer purchases.
- To calculate the total bill automatically.
- To update stock quantities after each sale.
- To reduce manual errors and save time.

1.2 Overview

The Retail Shop Management System is a Java-based application designed to simplify the daily activities of a retail shop. It helps manage product details, stock availability, sales transactions, and customer billing in an organized way.

The system reduces manual work, minimizes calculation errors, and makes it easier for shopkeepers to maintain accurate product and sales records. It uses Java programming

concepts such as classes, objects, methods, loops, and conditional statements to perform different operations efficiently.

The system can be further enhanced by integrating a database, developing a graphical user interface, and generating sales reports.

The application eliminates the need for manual calculations and helps users maintain accurate financial records. It can be developed either as a console-based program or as a graphical user interface using Java Swing.

This project demonstrates the practical use of Java programming in developing real-world applications related to financial management and digital transactions.

The Digital Wallet and Balance Management System includes the following main features:

1.3 Features

- 1. Product Management: Add, view, and update product details such as product ID, name, price, and quantity.
- 2. Stock Management: Check available stock and update product quantities after sales.
- 3. Billing System: Calculate the total bill automatically based on product prices and purchased quantities.
- 4. Sales Management: Record customer purchases and maintain sales details.
- 5. Product Availability: Check whether the required product is available before completing a sale.
- 6. Automatic Calculation: Reduce manual calculation errors by calculating bills using Java.
- 7. User-Friendly Menu: Provide a simple menu-driven interface to select and perform different operations.
- 8. Data Storage: Support saving and loading product records through file handling, if implemented.
- 9. Error Prevention: Validate product quantities to prevent selling more items than are available in stock.
- 10. Future Expansion: Allow integration with databases, graphical interfaces, and sales reports.

2.PROGRAM ANALYSIS

The Digital Wallet and Balance Management System manages financial transactions by storing and organizing user balance efficiently. It allows users to perform operations such as adding money, spending money, and checking their balance. The system automatically updates the balance after each transaction, ensuring accuracy and reducing manual effort.

It is developed using object-oriented programming concepts, making the system simple, efficient, and easy to use.

2.1 Problem Definition

In today's digital era, managing personal finances efficiently has become a major challenge. Many individuals still rely on manual methods such as remembering expenses or writing them down, which can lead to calculation errors, poor tracking, and mismanagement of money.Existing basic systems may allow users to store transaction details but often lack features such as automatic balance updates, proper tracking of spending, and easy access to current balance information. This creates confusion, especially when handling multiple transactions, and may lead to overspending or financial mismanagement.Therefore, there is a need for a smart system that can efficiently manage digital transactions and maintain accurate balance records. The Digital Wallet and Balance Management System is designed to address these issues by providing a simple and automated solution for managing money and tracking financial activities.

2.2 Project Overview

The **Digital Wallet and Balance Management System** is a Java-based project developed to help users manage their digital money effectively. The main objective of this system is to provide a simple platform where users can perform financial transactions and monitor their balance.

The application allows users to perform operations such as adding money, spending money, and checking their balance. The system automatically updates the balance after each transaction, ensuring accuracy and reliability.

The project is implemented using Core Java concepts such as object-oriented programming, classes, methods, and basic data handling. It is designed to be user-friendly and efficient, making it suitable for students and beginners.

Overall, this project provides a structured approach to managing financial transactions, helping users maintain control over their spending and improve financial discipline.

2.3 Scope and Limitations

Scope

The **Digital Wallet and Balance Management System** has a wide scope in managing personal financial transactions efficiently. It can be used by students, individuals, or anyone who wants to keep track of their money digitally.

The system supports basic operations such as adding money, spending money, and checking balance. It helps users maintain accurate financial records and reduces the chances of calculation errors.

The project can be further enhanced by adding advanced features such as:

- User authentication (login system)
- Transaction history
- Notifications and alerts
- Database integration
- Mobile application support

3.SYSTEM DESIGN

Digital Wallet and Balance Management System is designed as a simple and efficient system that helps users manage their financial transactions digitally. The system consists of three main components: the user interface, application logic, and data storage.The user interface allows users to interact with the system by performing operations such as adding money, spending money, and checking their balance. The application logic processes user inputs and performs necessary calculations to update the balance accurately. The data storage component maintains information related to the user's balance and transaction details.

The system is divided into modules including the wallet module, transaction management module, balance calculation module, and user interface module. Each module performs a specific function, ensuring smooth and efficient operation of the system.The design follows object-oriented programming principles, where the wallet is represented as an object with properties like balance and methods such as addMoney(), spendMoney(), and checkBalance(). The system ensures accurate and fast processing of transactions. Overall, the design is simple, modular, and easy to maintain and extend.

3.1 Modules

1. User Interface Module

Handles interaction between the user and the system. It allows users to:

- Add money
- Spend money
- Check balance

  Provides a simple menu-driven interface for easy usage.

2. Wallet Module

It stores the current balance and provides methods to manage it. Main functions include

- Storing balance
- Updating balance after transactions

3. Transaction Management Module

Manages all financial transactions such as:

- Adding money to the wallet
- Deducting money when spending

- Ensures all transactions are processed correctly.

4. Balance Calculation Module

Responsible for calculating and updating the balance after each transaction. Ensures:

- Accurate calculations
- No manual errors

5. Data Storage Module

Stores wallet information such as:

- Current balance
- Transaction details (optional)

Uses simple data structures or variables (basic version) and can be extended to databases in advanced versions.

6. Transaction History Module

Keeps track of all transactions performed by the user.
It can record:

- Amount added
- Amount spent
- Remaining balance

Helps users monitor their financial activity.\

4.METHODOLOGY

Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting the system.

For the Digital Wallet and Balance Management System, a structured development approach was followed to ensure that the system meets user requirements and functions efficiently.

The development process includes the following stages:

- Problem Analysis
- System Design
- Program Development
- Testing and Validation
- Result and Documentation

` 4.1 Problem Analysis

In this phase, the problem is studied and analyzed in detail. The requirements of the system are identified based on the need for managing digital transactions.

The main problems identified in manual financial tracking are:

- Manual calculations may lead to errors
- Difficulty in tracking expenses and balance
- Time-consuming process for managing transactions
- Lack of proper record maintenance

Based on these problems, system requirements were defined to develop an automated digital wallet system that simplifies financial management.

4.2 System Design

In this stage, the overall structure of the system is designed before coding begins.

The system is divided into modules such as:

- Add Money Module
- Spend Money Module
- Balance Check Module
- Transaction History Module
- Exit Module

The program flow and structure are designed to ensure that users can easily interact with the system.

4.3 Program Development

After designing the system, the program is implemented using the Java programming language.

The following steps were followed during development:

- Creation of classes and methods
- Implementation of menu-driven logic
- Writing code for transaction handling
- Handling user input
- Displaying balance and transaction details

Core Java concepts such as classes, objects, loops, conditional statements, and methods were used during development.

4.4 Testing and Validation

Testing ensures that the program works correctly and produces expected results.

Different test cases were used to verify system functionality.

Example Test Cases:

Test Case	Expected Result
Add money	Balance updated correctly
Spend money	Amount deducted correctly
Check balance	Correct balance displayed
Invalid input	Error handled properly

This phase ensures that the system runs smoothly without errors and provides accurate results.

4.5 Result and Documentation

After successful testing, the results of the system are documented.

The system displays:

- Updated balance after transactions
- Transaction details (if implemented)

Results are demonstrated through program execution and screenshots showing inputs and outputs.

Proper documentation is prepared to explain the system design, implementation, and working.

4.6 Algorithm

Algorithm for Digital Wallet System:

1. Start the program
2. Display menu options (Add Money, Spend Money, Check Balance, Exit)
3. Ask the user to choose an option
4. If user selects Add Money → enter amount → add to balance

5. If user selects Spend Money → enter amount → deduct from balance
6. If user selects Check Balance → display current balance
7. Repeat steps 3–6 until user selects Exit
8. End the program

4.7 Flowchart

Start

↓

Display Menu

↓

Select Option

↓

Add / Spend / Check Balance

↓

Update & Display Result

↓

Exit?

↓

End

5.SOURCE CODE

```
import java.util.ArrayList;
import java.util.Scanner;

class Product {
    int id;
    String name;
    double price;
    int quantity;

    Product(int id, String name, double price, int quantity) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.quantity = quantity;
    }

    void display() {
        System.out.println(
            id + "\t" + name + "\t" + price + "\t" + quantity
        );
    }
}

public class RetailShopManagement {
    static ArrayList<Product> products = new ArrayList<>();
    static Scanner sc = new Scanner(System.in);

    static Product findProduct(int id) {
        for (Product p : products) {
            if (p.id == id) {
                return p;
            }
        }
        return null;
    }
```

```
static void addProduct() {
    System.out.print("Enter Product ID: ");
    int id = sc.nextInt();
    sc.nextLine();

    if (findProduct(id) != null) {
        System.out.println("Product ID already exists!");
        return;
    }

    System.out.print("Enter Product Name: ");
    String name = sc.nextLine();

    System.out.print("Enter Price: ");
    double price = sc.nextDouble();

    System.out.print("Enter Quantity: ");
    int quantity = sc.nextInt();

    if (price < 0 || quantity < 0) {
        System.out.println("Invalid price or quantity!");
        return;
    }

    products.add(new Product(id, name, price, quantity));
    System.out.println("Product added successfully!");
}

static void viewProducts() {
    if (products.isEmpty()) {
        System.out.println("No products available.");
        return;
    }

    System.out.println("ID\tName\tPrice\tQuantity");
    for (Product p : products) {
        p.display();
    }
}

static void checkStock() {
    System.out.print("Enter Product ID: ");
    int id = sc.nextInt();
```

```
    Product p = findProduct(id);

    if (p != null) {
        System.out.println("Product: " + p.name);
        System.out.println("Available Stock: " + p.quantity);
    } else {
        System.out.println("Product not found.");
    }
}

static void sellProduct() {
    System.out.print("Enter Product ID: ");
    int id = sc.nextInt();

    Product p = findProduct(id);

    if (p == null) {
        System.out.println("Product not found.");
        return;
    }

    System.out.print("Enter Quantity to Sell: ");
    int qty = sc.nextInt();

    if (qty <= 0) {
        System.out.println("Enter a valid quantity.");
    } else if (qty > p.quantity) {
        System.out.println("Insufficient stock!");
    } else {
        double total = p.price * qty;
        p.quantity -= qty;

        System.out.println("------ CUSTOMER BILL ------");
        System.out.println("Product: " + p.name);
        System.out.println("Price: Rs. " + p.price);
        System.out.println("Quantity: " + qty);
        System.out.printf("Total Bill: Rs. %.2f%n", total);
        System.out.println("Sale completed successfully!");
    }
}

public static void main(String[] args) {
    int choice;
```

```
        do {
            System.out.println("\n=== RETAIL SHOP MANAGEMENT ===");
            System.out.println("1. Add Product");
            System.out.println("2. View Products");
            System.out.println("3. Check Stock");
            System.out.println("4. Sell Product");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");

            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    addProduct();
                    break;
                case 2:
                    viewProducts();
                    break;
                case 3:
                    checkStock();
                    break;
                case 4:
                    sellProduct();
                    break;
                case 5:
                    System.out.println("Thank you for using the system!");
                    break;
                default:
                    System.out.println("Invalid choice. Try again.");
            }
        } while (choice != 5);

        sc.close();
    }
}
```

6.INPUT AND OUTPUT SCREENS

Sample Run 1 – Add Money

Input:

==========

RETAIL SHOP MANAGEMENT SYSTEM

==========

1. Add Product

2. View Products

3. Check Stock

4. Sell Product

5. Exit

==========

Enter your choice:

Output:

ID Name Price Quantity

101 Notebook 50.00 100

7.DISCUSSION&CONCLUSION

7.1 Discussion

The Retail Shop Management System is developed using Java to simplify the daily activities of a retail shop. The system provides basic functions for managing product details, checking stock availability, processing sales, and calculating customer bills.

The program uses object-oriented programming concepts, classes, objects, methods, loops, and conditional statements to perform different operations. The menu-driven interface allows users to select the required operation easily.

The system calculates the total bill based on the product price and quantity purchased. It also updates the available stock after a successful sale and prevents transactions when the requested quantity exceeds the available stock.

7.2 Conclusion

The Retail Shop Management System provides a simple and effective solution for managing basic retail shop operations. It reduces manual calculations, improves stock tracking, and organizes product and sales information.

This project demonstrates the practical application of Java programming concepts in solving real-world problems. It also provides a foundation for developing more advanced retail management applications.

In the future, the system can be enhanced with database connectivity, a graphical user interface, user authentication, barcode scanning, and detailed sales reports. These improvements can make the application more reliable, secure, and suitable for larger retail businesses.

8.REFERENCES

The following resources were used during the development of this project:

- Herbert Schildt, Java: The Complete Reference, McGraw Hill Education.
- Oracle Official Documentation — Java Programming Language. https://docs.oracle.com/en/java/
- GeeksforGeeks — Java Programming Concepts and Examples. https://www.geeksforgeeks.org/java/
- W3Schools — Java Tutorial. https://www.w3schools.com/java/
- Java Documentation — Classes, Objects, Methods, and Collections.

- College Lecture Notes — Java Programming Lab and Object-Oriented Programming concepts.
- Laboratory Materials — Reference materials provided by the Department of B.Sc. Computer Science, KG College of Arts and Science.